

# Adaptive-Block-GS: Importance-Balanced Reconstruction for Large-Scale 3D Gaussian Splatting

Anonymous Authors

Paper under preparation for ICCIP 2026

Author names, affiliations, and contact information: **TBD**

**Abstract**—Large-scale novel-view synthesis and reconstruction are important for urban modeling, aerial inspection, and digital twins, but they impose substantially greater representation and optimization costs than object-scale settings. Although 3D Gaussian Splatting (3DGS) offers an explicit representation and fast rendering, its Gaussian count, memory demand, and training imbalance grow rapidly in city-, campus-, and aerial-scale scenes. Existing partitioned, hierarchical, and distributed methods improve scalability, yet fixed or poorly adapted partitions may assign blocks with very different complexity, RGB-only block optimization can produce unreliable geometry in weakly constrained regions, and directly merged blocks are generally not globally coordinated. We present Adaptive-Block-GS, a block-wise framework with three corresponding components. Importance-Guided Recursive Partitioning uses a coarse Gaussian importance field to adapt split decisions and selects supervising cameras through projection-based coarse screening followed by render-difference fine screening. Dual-Confidence Gated Depth Regularization applies aligned monocular depth only where the prior is cross-view consistent and the current Gaussian geometry is uncertain. Boundary-Constrained Attribute-Wise Global Fine-Tuning optimizes all attributes near block boundaries while restricting interior updates to appearance and opacity. We evaluate the framework on six scenes from MatrixCity, Mill19, and UrbanScene3D against 3DGS, VastGS, CityGS, Octree-GS, and Grendel-GS in terms of reconstruction quality, peak GPU memory, and training time.

**Index Terms**—3D Gaussian Splatting, large-scale reconstruction, novel-view synthesis, adaptive partitioning, geometry regularization

## 1 INTRODUCTION

Novel-view synthesis seeks a scene representation that can reproduce unobserved views from calibrated images. Classical image-based rendering interpolates densely captured rays or blends source images using proxy geometry [1], [2], often reconstructed by structure-from-motion and multiview stereo systems such as COLMAP [3]. Neural Radiance Fields (NeRFs) introduced continuous neural volumetric rendering and substantially improved view synthesis from sparse posed images [4]. Subsequent work accelerated optimization and rendering through multiscale sampling, sparse neural volumes, compact voxel fields, factorized tensors, and hash encodings [5]–[11]. More recently, 3D Gaussian Splatting (3DGS) replaced repeated neural-field queries with an explicit set of anisotropic Gaussian primitives and differentiable rasterization, enabling high-quality real-time rendering after optimization [12].

Scaling this representation to urban, campus, and aerial scenes exposes three coupled problems. First, the large spatial extent and nonuniform content make fixed or poorly adapted partitions assign blocks with very different Gaussian complexity. The resulting load imbalance can increase training time and leave difficult blocks under-optimized. Second, independently training each block only with RGB reconstruction loss can overfit the assigned views, especially in weakly textured or sparsely observed regions, producing floating primitives and inaccurate geometry. Third, direct concatenation of independently converged blocks is not a global optimum, but unconstrained full-scene retraining can overfit again and destabilize geometry that has already converged inside the blocks.

Large-scene NeRF systems address scale through spatial decomposition or multiple local models [13], [14]. Large-scene Gaussian methods inherit this divide-and-conquer strategy while retaining explicit rendering. VastGaussian (VastGS) uses progressive partitioning and visibility-aware camera selection [15]; CityGaussian (CityGS) trains a coarse representation, decomposes the scene into blocks, and uses coarse render differences for camera assignment [16]; Octree-GS provides a level-of-detail hierarchy for view-dependent rendering [17]; and Grendel scales optimization through distributed Gaussian and pixel parallelism [18]. These methods provide complementary partitioned, hierarchical, and distributed solutions. Our work focuses instead on adapting block complexity, selectively introducing depth supervision according to both prior reliability and geometric need, and coordinating a merged model without freely changing converged interior geometry.

We propose Adaptive-Block-GS, a block-wise reconstruction framework guided by a low-cost full-scene coarse Gaussian model. Its opacity-scale importance field determines recursive split decisions, while Gaussian projection provides coarse camera screening and render difference provides fine screening. During block optimization, aligned monocular depth is gated by cross-view prior confidence and rendered Gaussian depth uncertainty, so supervision concentrates on locations where the prior is reliable and the current geometry remains ambiguous. After core-owned block models are merged, a short global stage updates all Gaussian attributes inside boundary bands but only spherical-harmonic appearance and opacity in block interiors.

The main contributions of this work are summarized as

follows:

- 1) We propose Importance-Guided Recursive Partitioning, which uses coarse Gaussian opacity and scale to adapt block granularity and combines projection-based coarse screening with render-difference fine screening for camera assignment.
- 2) We introduce Dual-Confidence Gated Depth Regularization, which combines cross-view monocular-depth consistency with Gaussian ray-depth uncertainty to focus geometric supervision on reliable yet under-constrained regions.
- 3) We develop Boundary-Constrained Attribute-Wise Global Fine-Tuning, which permits full-parameter updates near block boundaries while limiting interior optimization to appearance and opacity to reduce the risk of destabilizing converged geometry.

## 2 RELATED WORK

### 2.1 Novel View Synthesis

Early novel-view synthesis methods represented the plenoptic function through dense ray samples or used reconstructed geometry to warp and blend source images [1], [2]. Their quality depends strongly on capture density, proxy accuracy, and robust visibility reasoning. NeRF instead learns a continuous mapping from 3D position and viewing direction to density and radiance and renders views by integrating samples along camera rays [4]. This formulation handles view-dependent effects and imperfect geometry implicitly, but its original dense sampling makes training and rendering costly.

Later neural fields improve efficiency by changing either sampling or representation. Mip-NeRF 360 introduces scene contraction and antialiased conical integration for unbounded scenes [5]. NSVF skips empty space using a sparse neural voxel structure [6], KiloNeRF distributes capacity across many small networks [7], and DVGO directly optimizes a voxel grid [8]. TensorRF factorizes a volumetric field into compact tensor components [9]; Plenoxels optimizes a sparse voxel grid without an MLP [10]; and Instant-NGP combines multiresolution hash encoding with compact networks [11]. These approaches progressively move computation from a large implicit network toward explicit, sparse, or factorized scene parameters.

3DGS completes this shift with anisotropic Gaussians carrying opacity and spherical-harmonic appearance [12]. Differentiable tile-based rasterization yields fast rendering and supports adaptive densification during training. Follow-up work improves aliasing and scale behavior [19] or organizes scene-dependent Gaussian anchors [20]. The explicit representation is convenient for editing, merging, and spatial selection, but its memory grows with the primitive count. Consequently, a scene that is much larger or more uneven than the object-scale benchmarks can exceed a practical device budget even though its final rasterization remains efficient.

### 2.2 Large-Scale Scene Reconstruction

Large-scale NeRF methods commonly divide a scene into spatially local models or progressively expand the represented

scale. BungeeNeRF trains a sequence of residual blocks for extreme multiscale city rendering [21]. Mega-NeRF partitions camera rays and scene content into cells that can be optimized separately, and its Mill19 Building and Rubble scenes expose the scale and appearance variation of aerial captures [13]. Block-NeRF composes many local NeRFs for city-scale driving sequences and models appearance variation across captures [14]. Such systems demonstrate that divide-and-conquer training can fit large environments, although composition, overlap, and resource allocation become part of the reconstruction problem.

The same principle has been adapted to explicit Gaussians. VastGS progressively partitions a scene, associates cameras and points using an airspace-aware visibility criterion, independently trains cells, and merges them for real-time rendering [15]. CityGS uses a coarse Gaussian scene to support block training and rendering-difference-based camera assignment, together with a scalable rendering representation [16]. Our work shares their coarse-to-fine and block-wise motivation, but uses the coarse Gaussian attributes directly to choose whether, where, and along which axis a block is split.

Hierarchical representations address another aspect of scale. Octree-GS organizes anchor-based Gaussians in a level-of-detail octree, selecting appropriate detail according to the view [17]. It is therefore complementary to a training decomposition whose main purpose is balancing representation load. Distributed systems instead expand the hardware budget. The system referred to as Grendel-GS distributes Gaussian parameters and pixels and studies scaling to very large primitive and image counts [18]. Because wall-clock time depends on GPU count, interconnect, per-device memory, image resolution, and batch construction, our experiments report distributed and single-device results with their complete hardware context rather than treating raw time as a hardware-independent measure.

Datasets have also expanded from bounded captures to city-scale imagery. MatrixCity provides synthetic city-scale trajectories and renderings [22], while UrbanScene3D contains real and synthetic large urban scenes for reconstruction and simulation [23]. Together with Mill19, these datasets cover aerial and ground-level observations, variable visibility, and nonuniform structural complexity, which are central to the proposed evaluation.

## 3 METHOD

### 3.1 Problem Formulation and Preliminaries

Given calibrated images  $\mathcal{I} = \{(I_c, \mathbf{K}_c, \mathbf{T}_c)\}_{c=1}^N$ , the goal is to optimize a Gaussian scene  $\mathcal{G}$  that renders the observed images and supports novel views. A primitive  $g_i$  contains center  $\boldsymbol{\mu}_i \in \mathbb{R}^3$ , scale  $\mathbf{s}_i \in \mathbb{R}_+^3$ , rotation  $\mathbf{R}_i \in SO(3)$ , opacity  $\alpha_i \in (0, 1)$ , and spherical-harmonic coefficients  $\mathbf{f}_i$ . Its world-space covariance is

$$\boldsymbol{\Sigma}_i = \mathbf{R}_i \text{diag}(\mathbf{s}_i^2) \mathbf{R}_i^\top. \quad (1)$$

## Teaser Placeholder

TODO: Large-scene input and camera trajectory → complete reconstruction → three enlarged local details.

Fig. 1. Teaser for large-scale reconstruction with Adaptive-Block-GS. The final figure will show the calibrated large-scene input, the unified Gaussian reconstruction, and matched close-ups that reveal distant structure and local geometric detail. No synthetic result is inserted at the drafting stage.

For camera  $c$ , the rasterizer projects  $\mu_i$  and approximates its screen-space covariance with the local projection Jacobian. After depth sorting, the color at pixel  $p$  is

$$\hat{I}_c(p) = \sum_{i=1}^M T_i(p) a_i(p) \mathbf{c}_i(\mathbf{d}_c), \quad T_i(p) = \prod_{j < i} (1 - a_j(p)), \quad (2)$$

where  $a_i(p)$  combines learned opacity with the projected 2D Gaussian density and  $\mathbf{c}_i(\mathbf{d}_c)$  is view-dependent color evaluated from the spherical harmonics along direction  $\mathbf{d}_c$ .

The standard photometric objective uses an  $L_1$  term and structural similarity [24]:

$$\mathcal{L}_{\text{photo}} = (1 - \lambda_s) \|\hat{I}_c - I_c\|_1 + \lambda_s (1 - \text{SSIM}(\hat{I}_c, I_c)). \quad (3)$$

### 3.2 Overview

The pipeline has three stages. First, a low-cost full-scene Gaussian model  $\mathcal{G}^c$  estimates the spatial distribution of representation load. Its attributes guide recursive partitioning, and its projected contribution supports camera assignment. Second, each block is initialized from  $\mathcal{G}^c$  and optimized independently with photometric supervision and dual-confidence gated monocular depth. Each block exports only core-owned Gaussians. Third, the core subsets are concatenated and globally fine-tuned with attribute-wise constraints determined by whether a Gaussian lies inside a block-boundary band.

### 3.3 Proposed Components

#### 3.3.1 Importance-Guided Recursive Partitioning:

**3.3.1.1 Coarse importance field:** We first train  $\mathcal{G}^c = \{g_i\}_{i=1}^M$  for a reduced full-scene budget. It is used as a spatial load estimate rather than the final reconstruction. The implemented importance of Gaussian  $i$  is

$$w_i = \alpha_i \bar{s}_i, \quad \bar{s}_i = \frac{1}{3} \sum_{k=1}^3 s_{i,k}. \quad (4)$$

Opacity reflects participation in image formation and mean scale approximates represented spatial extent. For scenes with large coordinate ranges, the centers can be mapped by the Mip-NeRF 360 contraction [5] using a configured axis-aligned bounding box. Partition operations then use contracted coordinates  $\tilde{\mu}_i$ , whereas rendering and optimization remain in world coordinates.

**3.3.1.2 Recursive split selection:** For node  $B$  with coarse indices  $\mathcal{J}_B$ , its total importance is

$$W(B) = \sum_{i \in \mathcal{J}_B} w_i. \quad (5)$$

If no absolute threshold is configured, the target leaf importance is the total scene importance divided by the requested maximum number of blocks. A node stops when it falls below the importance and optional density thresholds, or reaches the configured maximum depth, minimum point count, or spatial-size guard.

For each permitted axis  $a$ , we compute weighted variance

$$V_a(B) = \frac{1}{W(B)} \sum_{i \in \mathcal{J}_B} w_i (\tilde{\mu}_{i,a} - \bar{\mu}_a)^2, \quad \bar{\mu}_a = \frac{1}{W(B)} \sum_{i \in \mathcal{J}_B} w_i \tilde{\mu}_{i,a}. \quad (6)$$

The axis with maximum  $V_a$  is selected. Candidate positions are sampled from spatial quantiles rather than fixed at the box midpoint. For candidate  $t$ , the implementation evaluates

$$\begin{aligned} E(t) &= E_{\text{bal}}(t) + \lambda_{\text{bnd}} E_{\text{cut}}(t), \\ E_{\text{bal}}(t) &= \frac{|W(B_l(t)) - W(B_r(t))|}{W(B)}, \\ E_{\text{cut}}(t) &= \frac{1}{W(B)} \sum_{i: |\tilde{\mu}_{i,a} - t| \leq \delta_B} w_i, \end{aligned} \quad (7)$$

where  $B_l$  and  $B_r$  are the two candidate children and  $\delta_B$  is 2% of the occupied coordinate range on the selected axis. Minimizing (7) balances representation load and discourages cuts through locally important structure.

**3.3.1.3 Core ownership and camera assignment:** Each leaf has a nonoverlapping core box  $B^{\text{core}}$  and an optionally expanded training box  $B^{\text{exp}}$ . Expanded regions supply initialization or supervision context. At every save point, the implementation exports only Gaussians whose centers fall inside the core in partition coordinates:

$$\mathcal{G}_B^{\text{save}} = \{g_i \in \mathcal{G}_B \mid \tilde{\mu}_i \in B^{\text{core}}\}. \quad (8)$$

This ownership rule makes the subsequent merge a direct concatenation without duplicate overlap primitives.

Camera assignment follows a coarse-to-fine relevance test. We first project sampled coarse Gaussians from a block

### Method Overview Placeholder

TODO: Calibrated RGB images, camera poses, and sparse geometry  $\rightarrow$  coarse full-scene 3DGS  $\rightarrow$  importance-guided recursive blocks  
 $\rightarrow$  projection coarse screening and render-difference fine screening  $\rightarrow$  block optimization with a dual-confidence depth gate  
 $\rightarrow$  core-only merge  $\rightarrow$  boundary full-parameter and interior appearance-only fine-tuning  $\rightarrow$  unified Gaussian scene.

Fig. 2. Overview of Adaptive-Block-GS. Calibrated images, camera poses, and sparse geometry first produce a coarse Gaussian scene. Its opacity-scale importance guides recursive partitioning; Gaussian projection coarsely screens cameras before render-difference fine screening. Each block is initialized from the coarse scene and trained with RGB loss and dual-confidence gated monocular depth. Core-owned Gaussians are then merged. The final stage optimizes every attribute of boundary Gaussians while updating only spherical-harmonic appearance and opacity for interior Gaussians, yielding a unified scene.

into each calibrated view and retain cameras with sufficient projected support. We then use the block’s coarse render difference to fine-screen these candidates and keep views whose image contribution exceeds a configured threshold. If too few views remain, projected candidates are added according to inside-box status, coverage, and distance. Thus, cameras outside a block can still supervise structures that they observe, without evaluating fine render relevance for every image.

#### 3.3.2 Dual-Confidence Gated Depth Regularization:

**3.3.2.1 Aligned monocular depth:** For training view  $i$ , a pretrained monocular estimator [CITATION TODO: **monocular depth estimator**] produces depth  $D_i^P$ . Because monocular predictions have scale and shift ambiguity, we align them using sparse SfM observations or depth rendered by the coarse global model:

$$\tilde{D}_i^P(p) = s_i D_i^P(p) + t_i, \quad (9)$$

where  $s_i$  and  $t_i$  are the per-view alignment scale and shift. The block model renders depth  $\hat{D}_i$ . Invalid alignment, reprojection, and visibility support is excluded by a binary mask introduced below.

**3.3.2.2 Prior confidence:** Let  $\mathcal{N}(i)$  denote neighboring views that co-observe the current region, and let  $\tilde{D}_{j \rightarrow i}^P$  be the aligned depth of neighbor  $j$  reprojected into view  $i$ . We measure the relative cross-view disagreement at pixel  $p$  by

$$e_i^P(p) = \text{Med}_{j \in \mathcal{N}(i)} \frac{|\tilde{D}_i^P(p) - \tilde{D}_{j \rightarrow i}^P(p)|}{\tilde{D}_i^P(p) + \epsilon}. \quad (10)$$

The monocular prior confidence is then

$$C_i^P(p) = \exp\left(-\frac{e_i^P(p)}{\tau_P}\right), \quad (11)$$

where  $\tau_P > 0$  controls sensitivity to cross-view inconsistency. Reprojection uses visibility and depth checks so that occluded or unsupported correspondences do not contribute.

**3.3.2.3 Gaussian geometry uncertainty:** The depth prior should be emphasized not only when it is reliable, but also when the current Gaussian geometry remains uncertain. Let  $d_k$  be the camera-space depth of the  $k$ th Gaussian contributing

to pixel  $p$ , and let  $\omega_k(p)$  be its alpha-compositing weight. We compute the ray-depth variance

$$V_i^G(p) = \frac{\sum_k \omega_k(p) (d_k - \hat{D}_i(p))^2}{\sum_k \omega_k(p) + \epsilon}, \quad (12)$$

and map it to bounded uncertainty:

$$U_i^G(p) = 1 - \exp\left(-\frac{V_i^G(p)}{\tau_G}\right), \quad (13)$$

where  $\tau_G > 0$  sets the uncertainty scale. Large variance indicates that contributing Gaussians are dispersed along the ray, as can occur around thick surfaces, floating primitives, or geometrically ambiguous regions.

**3.3.2.4 Dual-confidence gate and loss:** Let  $M_i(p) \in \{0, 1\}$  combine valid aligned depth, cross-view reprojection support, and visibility checks. The final detached supervision weight is

$$W_i(p) = \text{sg}(M_i(p) C_i^P(p) U_i^G(p)), \quad (14)$$

where  $\text{sg}(\cdot)$  stops gradients through the gate. The multiplicative form gives high weight only when the prior is consistent across views and the current geometry is uncertain. We use a robust relative-depth objective

$$\mathcal{L}_{\text{depth}} = \frac{\sum_p W_i(p) \rho\left(\frac{\hat{D}_i(p) - \tilde{D}_i^P(p)}{\tilde{D}_i^P(p) + \epsilon}\right)}{\sum_p W_i(p) + \epsilon}, \quad (15)$$

where  $\rho(\cdot)$  is a robust penalty; its exact form and hyperparameters will be specified with the final implementation. The block objective is

$$\mathcal{L}_{\text{block}} = \mathcal{L}_{\text{photo}} + \lambda_d \mathcal{L}_{\text{depth}}. \quad (16)$$

**3.3.3 Boundary-Constrained Attribute-Wise Global Fine-Tuning:** Core-owned block models are concatenated in a fixed order,

$$\mathcal{G}^{\text{merge}} = \bigcup_{B \in \mathcal{B}} \mathcal{G}_B^{\text{save}}, \quad (17)$$

and the merge report records the contiguous Gaussian range originating from every block. For a source block  $B$ , Gaussian

### Partition–Train–Merge Placeholder

TODO: importance-guided recursive split; projection coarse screening and render-difference fine screening; dual-confidence depth gate; core-only export; boundary/interior attribute policy.

Fig. 3. Core mechanics of Adaptive-Block-GS. The final illustration will contrast midpoint or fixed-grid partitioning with importance-guided recursive cuts, show projection-based camera screening followed by render-difference fine screening, visualize the product of prior confidence and Gaussian geometry uncertainty, and distinguish full-parameter boundary updates from appearance-and-opacity-only interior updates.

$i$  is marked as a boundary primitive along configured partition axis  $a$  if

$$\min(\tilde{\mu}_{i,a} - B_a^{\min}, B_a^{\max} - \tilde{\mu}_{i,a}) \leq \beta(B_a^{\max} - B_a^{\min}), \quad (18)$$

where  $\beta$  is the relative band width. The union over selected axes gives boundary set  $\mathcal{G}_{\text{bnd}}$ ; all remaining primitives form  $\mathcal{G}_{\text{int}}$ .

The global stage applies an attribute-wise optimization policy. For every  $g_i \in \mathcal{G}_{\text{bnd}}$ , position  $\mu_i$ , scale  $s_i$ , rotation  $\mathbf{R}_i$ , opacity  $\alpha_i$ , and spherical-harmonic coefficients  $\mathbf{f}_i$  are all trainable. For every  $g_i \in \mathcal{G}_{\text{int}}$ , geometry is frozen while opacity and appearance remain trainable:

$$\nabla_{\mu_i, s_i, \mathbf{r}_i} \mathcal{L}_{\text{global}} = 0, \quad g_i \in \mathcal{G}_{\text{int}}, \quad (19)$$

$$\{\alpha_i, \mathbf{f}_i\} \text{ remain trainable}, \quad g_i \in \mathcal{G}_{\text{int}}. \quad (20)$$

Here  $\mathbf{r}_i$  denotes the optimized rotation parameterization of  $\mathbf{R}_i$ , and  $\mathcal{L}_{\text{global}} = \mathcal{L}_{\text{photo}}$ . This policy allows cross-block geometry and appearance to be coordinated near interfaces while preventing unconstrained global retraining from overfitting or destabilizing already converged interior geometry. Densification and pruning are disabled, so the source-block ranges and boundary mask remain fixed.

### 3.4 Training Objective and Implementation

The complete process is coarse training, partition construction, independent block optimization, core-only export, concatenation, and attribute-wise global fine-tuning. Block jobs can run independently on distinct GPUs, while each individual trainer samples one camera per iteration. Coarse and block stages retain the standard 3DGS densification schedule, whereas global fine-tuning disables densification and pruning. Exact iteration counts, learning rates, block limits, image resolutions, neighboring-view selection, reprojection checks, robust penalty, confidence temperatures, depth weight, and boundary-band width will be fixed per dataset and reported with the released configurations.

## 4 EXPERIMENTS

### 4.1 Experimental Setup

**Datasets.** We plan experiments on MatrixCity-Aerial [22], Mill19-Building and Mill19-Rubble [13], and the Residence, Sci-Art, and Campus scenes of UrbanScene3D [23]. These six scenes cover synthetic aerial imagery, real drone captures, and large urban environments. For each scene, we will report the number of training and test images, spatial extent, input resolution, and split protocol.

**Baselines.** We compare with the original 3DGS [12], VastGS [15], CityGS [16], Octree-GS [17], and Grendel-GS [18]. We will distinguish results reproduced under a matched hardware budget from results obtained with each method’s native or official configuration. Multi-GPU wall-clock time will not be interpreted as directly equivalent to single-GPU time.

**Metrics.** Image quality is measured using PSNR, SSIM [24], and LPIPS [25]. Resource evaluation reports end-to-end training time, peak allocated and reserved GPU memory where available, final Gaussian count, serialized model size, and rendering frames per second. We additionally record the per-block Gaussian count, assigned image count, training time, and peak memory to quantify load balance. Any metric not obtainable for a baseline will remain explicitly marked as unavailable or TODO rather than inferred.

**Reproducibility TODOs.** GPU model and number: **TBD**; memory per GPU: **TBD**; CPU and system memory: **TBD**; CUDA and PyTorch versions: **TBD**; image resolution: **TBD**; training iterations: **TBD**; train/test split: **TBD**; hyperparameters: **TBD**; baseline reproduction protocol: **TBD**; batch size for every method: **TBD**. For Grendel-GS, the final paper will explicitly list GPU type, GPU count, memory per GPU, batch size, input resolution, iteration count, interconnect if relevant, and total training time.

### 4.2 Comparison with State-of-the-Art Methods

Table I reserves the main quality comparison. Every value is intentionally left as **TBD**. Best and second-best formatting will be added only after the complete table is verified. Results will be grouped by matched hardware budget when feasible; native configurations will be reported separately if matching changes the intended operating regime.

Fig. 4 reserves matched views and crops for the six-scene qualitative comparison. It is placed near the quantitative results so that visual evidence can be read together with the quality metrics.

### 4.3 Efficiency and Scalability Analysis

Table II records the variables needed to interpret resource cost. End-to-end time includes all method-required stages, including coarse training, partitioning, block training, merging, and global refinement for our method. We will also break down these stages in the supplementary material or final main paper if space permits. Peak memory is reported per GPU and together with the number of GPUs; model size and Gaussian count refer to the final representation used for evaluation.

TABLE I  
NOVEL-VIEW SYNTHESIS QUALITY ON SIX LARGE-SCALE SCENES. NO METHOD IS PRE-MARKED AS BEST. HIGHER IS BETTER FOR PSNR AND SSIM;  
LOWER IS BETTER FOR LPIPS.

| Metric             | Dataset           | 3DGS | VastGS | CityGS | Octree-GS | Grendel-GS | Ours |
|--------------------|-------------------|------|--------|--------|-----------|------------|------|
| PSNR $\uparrow$    | MatrixCity-Aerial | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | Mill19-Building   | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | Mill19-Rubble     | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | US3D-Residence    | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | US3D-Sci-Art      | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | US3D-Campus       | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
| SSIM $\uparrow$    | MatrixCity-Aerial | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | Mill19-Building   | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | Mill19-Rubble     | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | US3D-Residence    | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | US3D-Sci-Art      | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | US3D-Campus       | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
| LPIPS $\downarrow$ | MatrixCity-Aerial | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | Mill19-Building   | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | Mill19-Rubble     | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | US3D-Residence    | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | US3D-Sci-Art      | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |
|                    | US3D-Campus       | TBD  | TBD    | TBD    | TBD       | TBD        | TBD  |

### Six-Dataset Qualitative Comparison Placeholder

TODO: Rows for MatrixCity-Aerial, Mill19-Building, Mill19-Rubble, Residence, Sci-Art, and Campus; columns for reference, 3DGS, VastGS, CityGS, Octree-GS, Grendel-GS, and Ours; include matched zoom boxes.

Fig. 4. Planned qualitative comparison on the six evaluation scenes. Every method will use the same camera and crop coordinates. Enlargements will emphasize geometric boundaries, distant detail, floating artifacts, and transitions near reconstructed block boundaries. Missing official outputs will be labeled rather than replaced by incomparable images.

To isolate scale, we will vary the target block count while preserving the image set and common optimization budget. Table III reports reconstruction quality together with the maximum and coefficient of variation of per-block Gaussian count, time, and memory. Fig. 5 will plot these quantities against scene size and block count.

#### 4.4 Ablation Study

The component ablation in Table IV follows the three proposed components cumulatively. The base system uses fixed block training and core-only merging. Subsequent rows add importance-guided recursive partitioning, dual-confidence

gated depth regularization, and boundary-constrained attribute-wise global fine-tuning. We will report both quality and resource effects after every component and will not infer improvements before the experiments are complete.

We will further diagnose each component through: (i) opacity versus opacity-scale importance; (ii) midpoint versus weighted split selection; (iii) projection-only camera assignment versus projection plus render-difference screening; (iv) prior confidence only, Gaussian geometry uncertainty only, and their multiplicative dual gate; (v) no global fine-tuning, unconstrained global fine-tuning, and boundary-constrained

TABLE II  
TRAINING AND RESOURCE COMPARISON. HARDWARE CONTEXT IS PART OF EACH RESULT. NATIVE-CONFIG AND MATCHED-BUDGET MEASUREMENTS WILL BE SEPARATED IN THE COMPLETED TABLE.

| Method     | GPU        | #GPU       | Mem./GPU   | Batch      | Resolution | Time       | Gaussians  | FPS        |
|------------|------------|------------|------------|------------|------------|------------|------------|------------|
| 3DGS       | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| VastGS     | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| CityGS     | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| Octree-GS  | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| Grendel-GS | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| Ours       | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | 1/block    | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |

TODO: add peak allocated/reserved memory and serialized model size, either as columns or a second panel after actual ranges are known.

TABLE III  
SCALABILITY AND LOAD BALANCE ON A REPRESENTATIVE LARGE SCENE.

| Blocks | PSNR       | Total time | Peak mem.  | Load CV    |
|--------|------------|------------|------------|------------|
| 4      | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| 8      | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| 16     | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| 32     | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |

TABLE IV  
COMPONENT ABLATION ON A REPRESENTATIVE SCENE.

| Partition | Dual gate | Boundary FT | PSNR       | LPIPS      | Time       | Mem.       |
|-----------|-----------|-------------|------------|------------|------------|------------|
| –         | –         | –           | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| ✓         | –         | –           | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| ✓         | ✓         | –           | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |
| ✓         | ✓         | ✓           | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> | <b>TBD</b> |

### Scalability Curves Placeholder

TODO: scene size or block count versus end-to-end time, per-GPU peak memory, Gaussian count, and per-block load dispersion.

Fig. 5. Planned scalability analysis. Curves will distinguish total compute from wall-clock time and will include GPU count and per-device memory. Load dispersion will be measured from block-level Gaussian counts, training times, and peak memory rather than inferred from block area.

attribute-wise fine-tuning; (vi) boundary-band width; and (vii) target block count. These tests will quantify effects on reconstruction quality, training time, peak memory, per-block load dispersion, and stability as scene scale increases.

#### 4.5 Qualitative Analysis

Fig. 4 will show identical crops from all six datasets. The final discussion will focus on facade edges, roofs, thin structures, distant regions, floaters, and block boundaries. Claims will be tied to visible evidence and the corresponding ablations; no qualitative ranking is asserted in the current draft.

## 5 CONCLUSION

We presented Adaptive-Block-GS, a large-scale 3D Gaussian Splatting framework organized around three coupled components. Coarse opacity-scale importance guides recursive partitioning, followed by projection-based coarse screening and render-difference fine screening of cameras. Dual-confidence gated depth regularization combines cross-view prior consistency with Gaussian ray-depth uncertainty during block training. After core-only merging, boundary-constrained

attribute-wise global fine-tuning updates every attribute near block interfaces while limiting interior changes to appearance and opacity. The planned six-scene evaluation will quantify reconstruction quality, peak GPU memory, training time, load balance, and the behavior of each component without presuming the final ranking.

## REFERENCES

- [1] M. Levoy and P. Hanrahan, “Light field rendering,” in *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*, 1996, pp. 31–42.
- [2] S. J. Gortler, R. Grzeszczuk, R. Szeliski, and M. F. Cohen, “The lumigraph,” in *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*, 1996, pp. 43–54.
- [3] J. L. Schönberger and J.-M. Frahm, “Structure-from-motion revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [4] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *European Conference on Computer Vision*, 2020, pp. 405–421.
- [5] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, “Mip-NeRF 360: Unbounded anti-aliased neural radiance fields,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [6] L. Liu, J. Gu, K. Z. Lin, T.-S. Chua, and C. Theobalt, “Neural sparse voxel fields,” in *Advances in Neural Information Processing Systems*, 2020.
- [7] C. Reiser, S. Peng, Y. Liao, and A. Geiger, “KiloNeRF: Speeding up neural radiance fields with thousands of tiny MLPs,” in *IEEE/CVF International Conference on Computer Vision*, 2021.
- [8] C. Sun, M. Sun, and H.-T. Chen, “Direct voxel grid optimization: Super-fast convergence for radiance fields reconstruction,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [9] A. Chen, Z. Xu, A. Geiger, J. Yu, and H. Su, “TensorRF: Tensorial radiance fields,” in *European Conference on Computer Vision*, 2022.
- [10] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, “Plenoxels: Radiance fields without neural networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [11] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM Transactions on Graphics*, vol. 41, no. 4, 2022.

- [12] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.
- [13] H. Turki, D. Ramanan, and M. Satyanarayanan, “Mega-NeRF: Scalable construction of large-scale NeRFs for virtual fly-throughs,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [14] M. Tancik, V. Casser, X. Yan, S. Pradhan, B. Mildenhall, P. P. Srinivasan, J. T. Barron, and H. Kretzschmar, “Block-NeRF: Scalable large scene neural view synthesis,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [15] J. Lin, Z. Li, X. Tang, J. Liu, S. Liu, J. Liu, Y. Lu, X. Wu, S. Xu, Y. Yan, and W. Yang, “VastGaussian: Vast 3d gaussians for large scene reconstruction,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [16] Y. Liu, H. Guan, C. Luo, L. Fan, N. Wang, J. Peng, and Z. Zhang, “CityGaussian: Real-time high-quality large-scale scene rendering with gaussians,” in *European Conference on Computer Vision*, 2024.
- [17] K. Ren, L. Jiang, T. Lu, M. Yu, L. Xu, Z. Ni, and B. Dai, “Octree-GS: Towards consistent real-time rendering with LOD-structured 3d gaussians,” 2024.
- [18] H. Zhao, H. Weng, D. Lu, A. Li, J. Li, A. Panda, and S. Xie, “On scaling up 3d gaussian splatting training,” in *International Conference on Learning Representations*, 2025.
- [19] Z. Yu, A. Chen, B. Huang, T. Sattler, and A. Geiger, “Mip-splatting: Alias-free 3d gaussian splatting,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [20] T. Lu, M. Yu, L. Xu, Y. Xiangli, L. Wang, D. Lin, and B. Dai, “Scaffold-gs: Structured 3d gaussians for view-adaptive rendering,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [21] Y. Xiangli, L. Xu, X. Pan, N. Zhao, A. Rao, C. Theobalt, B. Dai, and D. Lin, “BungeeNeRF: Progressive neural radiance field for extreme multi-scale scene rendering,” in *European Conference on Computer Vision*, 2022.
- [22] Y. Li, L. Jiang, L. Xu, Y. Xiangli, Z. Wang, D. Lin, and B. Dai, “MatrixCity: A large-scale city dataset for city-scale neural rendering and beyond,” in *IEEE/CVF International Conference on Computer Vision*, 2023.
- [23] L. Lin, Y. Liu, Y. Hu, X. Yan, K. Xie, and H. Huang, “Capturing, reconstructing, and simulating: The UrbanScene3D dataset,” in *European Conference on Computer Vision*, 2022.
- [24] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [25] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.